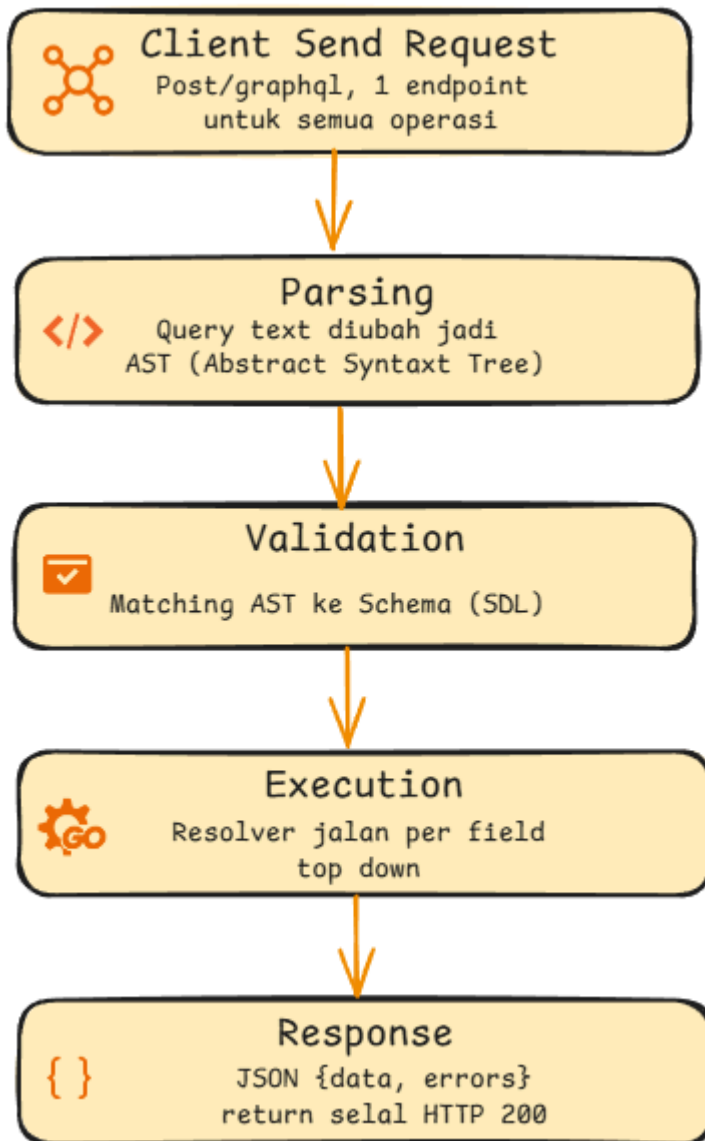


2. Workflow GraphQL

1. Diagram workflow GraphQL :



How it works :

1. Client Send Request

```
// HTTP POST http://localhost:3002/graphql
{
  "query": "query GetEmployeeData($empId: Int!) { employee(id: $empId) {
lastName job { jobTitle } } }",
  "variables": {
    "empId": 1
  }
}
```

2. Parsing

```
// Secara internal, string query diubah oleh GraphQL parser menjadi AST
Object seperti ini:
const ast = {
  kind: 'Document',
  definitions: [
    {
      kind: 'OperationDefinition',
      operation: 'query',
      name: { kind: 'Name', value: 'GetEmployeeData' },
      selectionSet: {
        kind: 'SelectionSet',
        selections: [
          {
            kind: 'Field',
            name: { kind: 'Name', value: 'employee' },
            arguments: [{ name: 'id', value: { kind: 'Variable', name:
'empId' } } ]},
          { kind: 'Field', name: { kind: 'Name', value: 'lastName' } },
          { kind: 'Field', name: { kind: 'Name', value: 'job' } },
        ],
      },
    },
    ...
  ],
};
```

3. Validation

```
// employees.typeDefs.ts
export const employeeTypeDefs = `
  type Job {
    jobId: Int!
    jobTitle: String!
  }

  type Employee {
    employeeId: Int!
    lastName: String!
    job: Job
  }

  type Query {
    employee(id: Int!): Employee
  }
`;
```

```
// JIKA Client mengirim: { employee(id: "SATU") { salaryInfo } }
// Validasi GAGAL karena "id" harus Int (bukan String) dan field
"salaryInfo" tidak ada di SDL!
```

4. Execution

```
// employees.resolvers.ts
export const employeeResolvers = {
  // 1. Root Resolver dieksekusi PERTAMA (Top)
  Query: {
    employee: async (_parent: unknown, args: { id: number }) => {
      // Memanggil service untuk query ke DB via Drizzle
      return await employeeService.getEmployeeById(args.id);[cite: 4]
    },
  },

  // 2. Field Resolver dieksekusi KEMUDIAN (Down - Hanya jika field "job"
  // diminta client)
  Employee: {
    job: (parent: { jobId: number }, _args: unknown, ctx: MercuriusContext)
    => {
      // Menggunakan DataLoader untuk mengambil data relasi job
      return ctx.loaders.jobLoader.load(parent.jobId);[cite: 4]
    },
  },
};
```

5. Response

```
// Response Sukses (HTTP 200 OK)
{
  "data": {
    "employee": {
      "lastName": "King",
      "job": {
        "jobTitle": "President"
      }
    }
  }
}

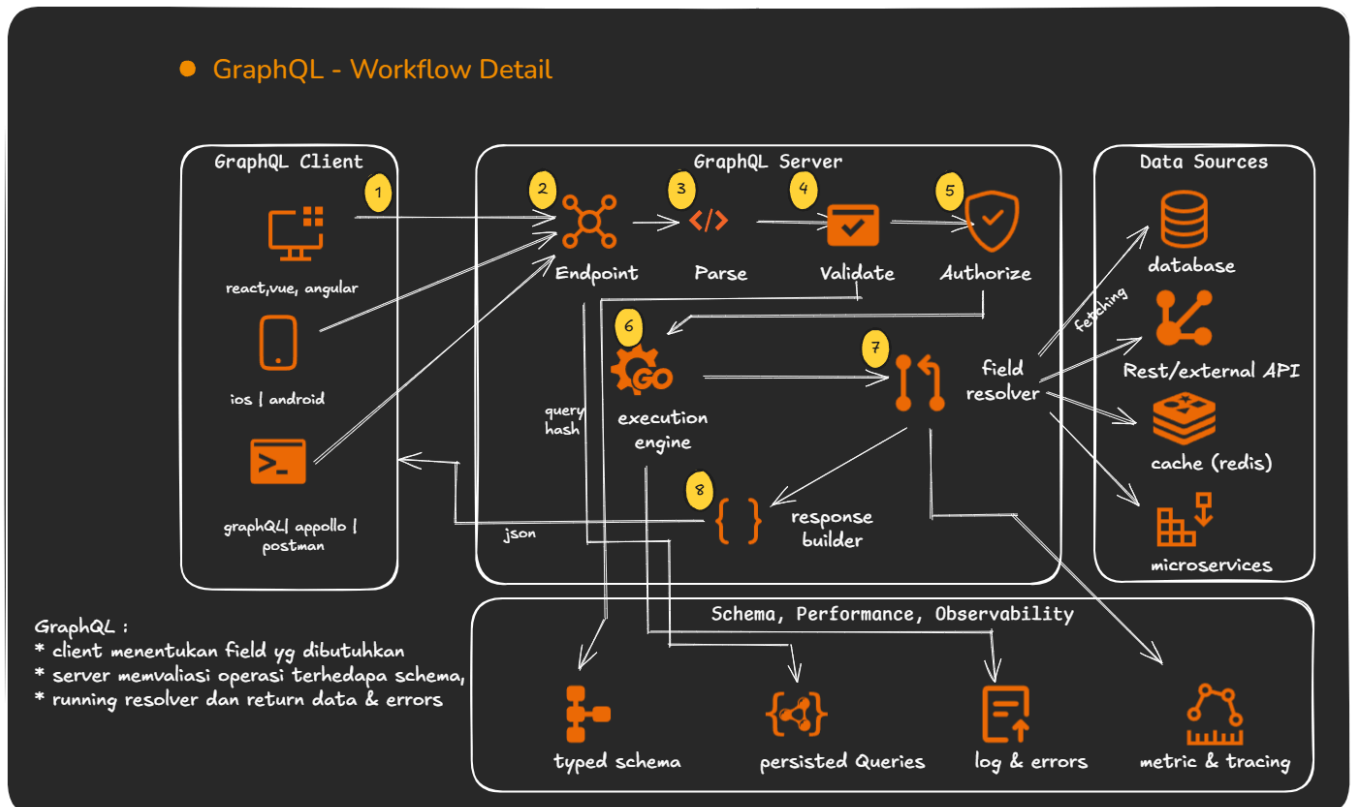
// Jika Terjadi Error (Tetap HTTP 200 OK)
{
  "data": {
    "employee": null
  },
  "errors": [
    {
      "message": "Employee with ID 999 not found",
    }
  ]
}
```

```

    "path": ["employee"],
    "extensions": {
      "code": "NOT_FOUND"
    }
  }
]
}

```

3. GraphQL Workflow With Detail



How it works :

- 1. Client Kirim Request :** Web app (React/Vue/Angular), mobile app (iOS/Android), atau tool dev (GraphiQL/Postman/Apollo) kirim POST ke satu endpoint yang sama — tidak ada endpoint berbeda per jenis client.
- 2. Endpoint diterima :** Single main entry (di project yg dibuat: Mercurius yang di-register di app.ts) menerima body { query, variables } dari client manapun.
- 3. Parse - Query jadi AST :** Teks query diubah jadi Abstract Syntax Tree (pohon sintaks) supaya bisa diproses secara terstruktur, bukan diparsing sebagai string.
- 4. Validate - Matching ke Schema :** AST dicek terhadap Typed Schema (garis panah ke bawah dari Parse). Kalau ada field yang tidak dikenal atau tipe salah, request ditolak di point ini — tahap berikutnya tidak pernah jalan.
- 5. Authorize - Check Identitas & Ijin :** Setelah lolos validasi struktur, dicek apakah user/token yang mengirim request punya izin untuk operasi ini. Ini gate terakhir sebelum data beneran disentuh.

6. **Execution Engine** - Jalankan operation tree : Mengatur urutan eksekusi resolver — root field dulu, baru field turunannya — dan bisa menjalankan beberapa field secara paralel kalau tidak saling bergantung. Query hash-nya juga dicatat di sini untuk Persisted Queries.
7. **Field Resolver ambil data** : Resolver per-field benar-benar mengambil data dari Data Sources — bisa dari database, REST/external API, cache (Redis), atau microservices lain, tergantung field apa yang diminta. Aktivitas ini juga yang dicatat ke Metric & Tracing.
8. **Response Builder** susun hasil : Semua hasil resolver digabung jadi satu objek JSON { data, errors }, dicatat ke Log & Errors, lalu dikirim balik ke client yang sama persis lewat endpoint yang sama (garis panah 'json' kembali ke kiri).